Lab2-内存读写指令

- 姓名：戴益阳
- 专业：计算机科学与技术
- 学号：24300240114

- 实验时间：2026/3/17 ~ 2025/3/29
- 指导老师：陈辰
- 本次实验，我完成了所有内容。

目录

- Lab2-内存读写指令
  - 目录
  - 实验目的
  - 实验内容
    - 1. RISC-V指令格式
    - 2. 解决数据冲突问题（Data Hazard）
    - 3. IO总线
    - 4. 使用verilator工具实现CPU的设计
  - 实验结果
  - 遇到的问题及解决办法
  - 实验心得
  - 其他备注

实验目的

1. 理解RISC-V指令集的编码格式与执行流程，掌握指令分类及其控制信号生成机制。
2. 掌握流水线CPU的数据通路与控制单元设计方法，在实现转发的基础上增加阻塞。
3. 掌握带有IO总线的CPU的设计方法，实现读写内存时CPU与IO总线的交互。
4. 掌握使用gtkwave工具观察波形图，并进行代码调试

实验内容

本次实验要求在上次实验的基础上，继续实现能够执行如下指令的带有IO总线的64位RISC-V流水线CPU：

内存读写：

lb, lh, lw, ld, lbu, lhu, lwu

sb, sh, sw, sd

lui

1. RISC-V指令格式

通过查表可知，本次实验要实现的指令格式如下：

```
imm[11:0] rs1 000 rd 0000011 I lb
imm[11:0] rs1 001 rd 0000011 I lh
imm[11:0] rs1 010 rd 0000011 I lw
imm[11:0] rs1 011 rd 0000011 I ld
imm[11:0] rs1 100 rd 0000011 I lbu
imm[11:0] rs1 101 rd 0000011 I lhu
imm[11:0] rs1 110 rd 0000011 I lwu

imm[11:5] rs2 rs1 000 imm[4:0] 0100011 S sb
imm[11:5] rs2 rs1 001 imm[4:0] 0100011 S sh
imm[11:5] rs2 rs1 010 imm[4:0] 0100011 S sw
imm[11:5] rs2 rs1 011 imm[4:0] 0100011 S sd

imm[31:12] rd 0110111 U lui
```

其中，I型指令中load类指令的计算方式为：

rd = (rs1 + imm)

S型指令的计算方式为：

(rs1 + imm) = rs2

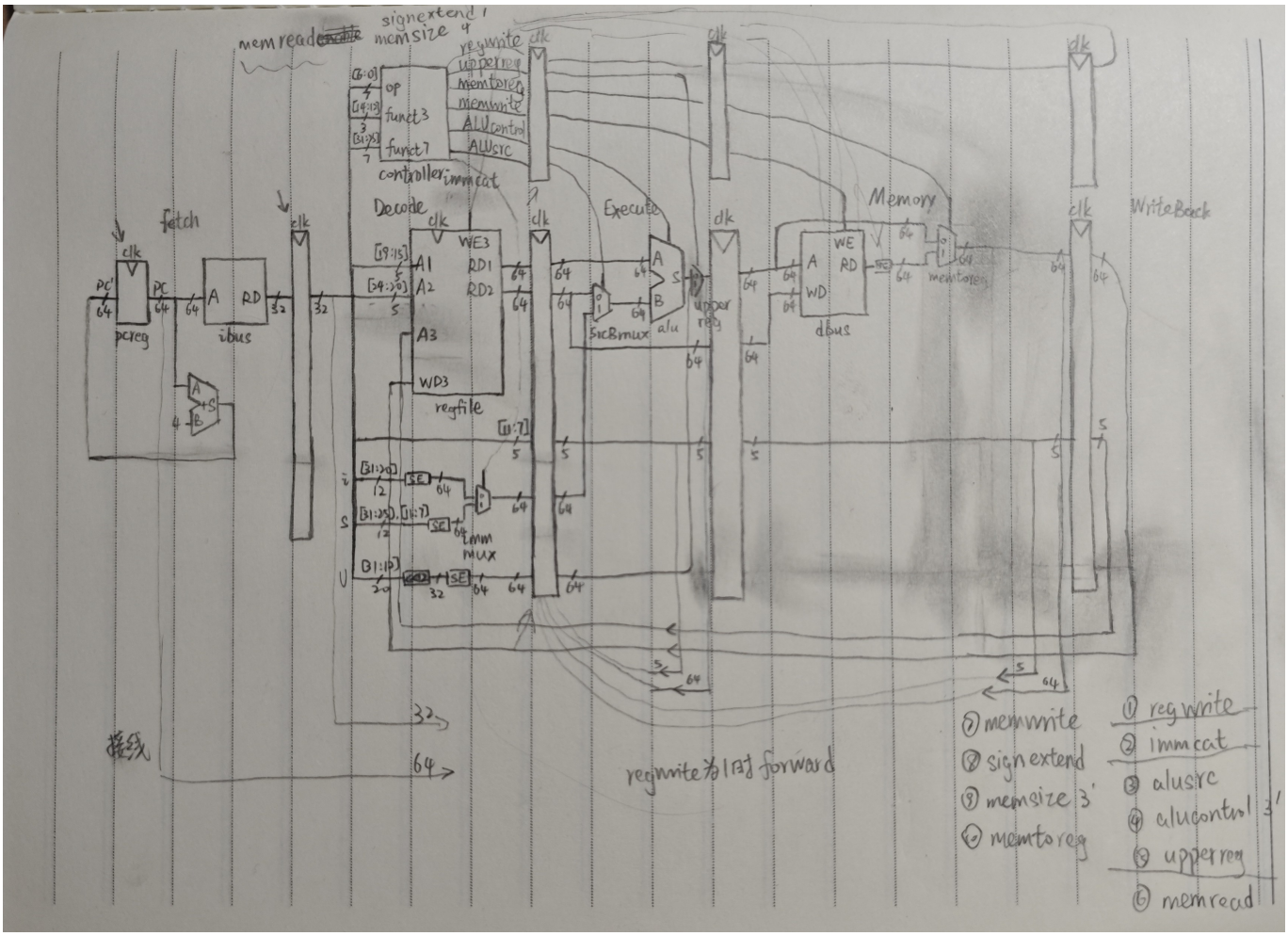
U型指令的计算方式为：

rd = imm << 12

2. 解决数据冲突问题（Data Hazard）

本次实验中由于增加了内存读写的指令，会出现load-use的情况，因此必须增加阻塞（Stall）来解决数据冲突问题（Data Hazard），即execute模块中的指令为load类指令并且load指令的目的寄存器为decode模块中的一个源寄存器时，需要阻塞一个周期。

最后得到的设计图如下：



3. IO总线

本次实验要实现带有IO总线的CPU，要在上次实现了与ibus交互的基础上实现与dbus的交互。dbus与ibus的不同之处在于

1. dbus同时要求读和写，这意味着同时需要读使能和写使能的控制信号，还需要注意这两个信号是随着step的步进信号和step同时到达memory模块的，因此需要晚一个时钟周期才能向dbus发出读或写的请求。

2. dbus有对齐要求，无论写还是读都要求地址向下取整到8的倍数，即看作地址的低3位置为0，对齐后的地址和原本地址的差需要用于实现strobe和data信号的对齐。

4. 使用verilator工具实现CPU的设计

核心代码有：

stall部分：

```
// stall

logic stall;

logic [4:0] readAddr1, readAddr2;

assign readAddr1 = instrD[19:15];
```

```
assign readAddr2 = instrD[24:20];

assign stall = memreadE & (writeRegE != 0)
                & ((writeRegE == readAddr1) || (writeRegE == readAddr2));

...

stallreg #(13) cregDE(clk, reset, step, stall,
                {regwriteD, alusrcD, alucontrolD, upperregD,
                memreadD, memwriteD, signextendD, memsizeD, memtoregD},
                {regwriteE, alusrcE, alucontrolE, upperregE,
                memreadE, memwriteE, signextendE, memsizeE, memtoregE});

// forward

logic [63:0] true_readData1D, true_readData2D;

logic [4:0] sreg1, sreg2;

assign sreg2 = instrD[24:20];
assign sreg1 = instrD[19:15];

always_comb begin
    true_readData1D = readData1D;
    true_readData2D = readData2D;
    if (regwriteM && writeRegM != 0) begin
        if (writeRegM == sreg1) true_readData1D = memresultM;
        if (writeRegM == sreg2) true_readData2D = memresultM;
    end
    if (regwriteE && writeRegE != 0) begin
        if (writeRegE == sreg1) true_readData1D = aluresultE;
        if (writeRegE == sreg2) true_readData2D = aluresultE;
    end
end

stallreg #(357) dregDE(clk, reset, step, stall,
                {true_readData1D, true_readData2D, writeRegD, seimmD, seuimmD,
pcD, instrD},
                {readData1E, readData2E, writeRegE, seimmE, seuimmE, pcE,
instrE});
```

memory部分：

```
`ifdef VERILATOR
`include "include/common.sv"
`include "src/mux2.sv"
`endif

module memory import common::*;(
    input  logic clk, reset,
    input  logic step, // 这个信号用来同步整个 CPU 的时序，当其为 1 时，整个 CPU 流水
```

```
线向前移动一个指令。
    output logic memory_ok, // 表示当前模块是否已经准备好接受下一条指令了
    // 实际上 step = fetch_ok & decode_ok & execute_ok & memory_ok & writeback_ok;
也就是说，只有当五个阶段都准备好接受下一条指令了，step 才会为 1。
    input  logic re,
    input  logic we,
    input  logic signextend,
    input  msize_t size,
    input  logic memtoreg,
    input  logic [63:0] aluresult,
    input  logic [63:0] writedata,
    input  dbus_resp_t dbus_resp,
    output dbus_req_t  dbus_req,
    output logic [63:0] result
    );

logic [2:0] offset;

assign offset = aluresult[2:0];

logic [3:0] sizenum;

logic [7:0] strobe;

logic [63:0] aligned_writedata;

always_comb begin
    case (size)
        MSIZE1: sizenum = 1;
        MSIZE2: sizenum = 2;
        MSIZE4: sizenum = 4;
        MSIZE8: sizenum = 8;
        default: sizenum = 8;
    endcase
    strobe = 8'b0;
    for (int i = 0; i < sizenum; i++) begin
        if (32'(offset) + i < 8)
            strobe[32'(offset) + i] = 1'b1;
    end
    aligned_writedata = writedata << (offset * 8);
end

logic busy;

logic [63:0] readdata;

always_ff @(posedge clk) begin
    if (reset) begin
        memory_ok <= 1;
        dbus_req.valid <= 0;
        dbus_req.addr <= 0;
        dbus_req.strobe <= 0;
        dbus_req.data <= 0;
        busy <= 0;
```

```
    end else if (step) begin
        memory_ok <= 0; // 先把 execute_ok 置为 0，表示我们正在处理当前指令，还没有准备好接受下一条指令了。
        busy <= 1;
    end else if (busy) begin
        // read
        if (re) begin
            dbus_req.valid <= 1; // 置为 1，表示我们要求读数据了。
            dbus_req.addr <= aluresult;
            dbus_req.size <= size;
            dbus_req.strobe <= 8'b0;
            dbus_req.data <= 0;
            busy <= 0;
        end

        //write
        else if (we) begin
            dbus_req.valid <= 1; // 置为 1，表示我们要求写数据了。
            dbus_req.addr <= aluresult;
            dbus_req.size <= size;
            dbus_req.strobe <= strobe;
            dbus_req.data <= aligned_writedata;
            busy <= 0;
        end

        // no read and write
        else begin
            busy <= 0;
        end
    end
    else begin
        // 这里对应：要么我们还没执行好指令，要么我们执行好指令了，在等其他模块
        if (memory_ok) begin
            // 在等其他模块
        end else begin
            if (re) begin
                if (dbus_resp.data_ok & dbus_resp.addr_ok) begin
                    readdata <= dbus_resp.data;
                    dbus_req.valid <= 0;
                    memory_ok <= 1;
                end
            end

            else if (we) begin
                if (dbus_resp.data_ok & dbus_resp.addr_ok) begin
                    dbus_req.valid <= 0;
                    memory_ok <= 1;
                end
            end

            else if (!re & !we) begin
                memory_ok <= 1;
            end
        end
```

```
    end
end

logic [63:0] aligned_readdata;

assign aligned_readdata = readdata >> (offset * 8);

logic [63:0] memresult;

always_comb begin
    case (size)
        MSIZE1:
            if (signextend) memresult = {{56{aligned_readdata[7]}},
aligned_readdata[7:0]};
            else memresult = {{56{1'b0}}, aligned_readdata[7:0]};
        MSIZE2:
            if (signextend) memresult = {{48{aligned_readdata[15]}},
aligned_readdata[15:0]};
            else memresult = {{48{1'b0}}, aligned_readdata[15:0]};
        MSIZE4:
            if (signextend) memresult = {{32{aligned_readdata[31]}},
aligned_readdata[31:0]};
            else memresult = {{32{1'b0}}, aligned_readdata[31:0]};
        MSIZE8: memresult = aligned_readdata;
        default: memresult = aligned_readdata;
    endcase
end

mux2 memtoregmux(aluresult, memresult, memtoreg, result);

endmodule
```

实验结果

```
TEST= ./build/emu --diff /home/dyy/arch_2026/26-Arch/ready-to-run/riscv64-nemu-interpreter-so -i ./ready-to-run/lab2/lab2-test.bin  || true
Emu compiled at Mar 10 2026, 17:16:55
The image is ./ready-to-run/lab2/lab2-test.bin
Using simulated 256MB RAM
Using /home/dyy/arch_2026/26-Arch/ready-to-run/riscv64-nemu-interpreter-so for difftest
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'clint' at [0x38000000, 0x3800ffff]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
The first instruction of core 0 has commited. Difftest enabled.
[WARNING] difftest store queue overflow
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x000000008001fffc
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 8160 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 32767
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 4015563 instr/s
Program execution has ended. To restart the program, exit NEMU and run again.
sh: 1: spike-dasm: not found
```

遇到的问题及解决办法

1. 遇到问题： 转发和阻塞同时使用

   解决方案：需要修改上次写的转发的对象，并且要注意阻塞信号的持续时间

2. 遇到问题： dbus的读使能和写使能信号是随着step的步进信号和step同时到达memory模块的

解决方案：需要晚一个时钟周期才能开始向dbus发出到底是读还是写的请求

3. 遇到问题： 仿真时报错：mux2.sv重复定义

   解决方案：文件头尾增加了宏

实验心得

这次实验让我学会了带有IO总线的流水线CPU的实现方法，掌握了在verilator环境和Difftest要求下的实现方法，以及使用gtkwave工具查看波形图逐步调试，为后续增加更多指令奠定了基础。

其他备注

无